

Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

Run 1: 1.55638112

Run 2: 1.99014344

Run 3: 1.77793000

Run 4: 1.22511072

Run 5: 1.36084016

Part 2: The Synchronization Trap

- Fix the race condition by adding `synchronized` to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

Plain Single-Threaded (Serial): 452.447 ms

Synchronized (4 Threads): 6781.620 ms (медленнее примерно в **15 раз**)

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads on their machine with \$100,000,000\$ iterations.
- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (T1/TN)	Efficiency (Speedup/T)
1	916.240 (<i>baseline</i>)	1.0x	100%
2	988.785	0.93x	46.33%
4	1101.647	0.83x	20.79%
8	1150.000	0.80x	10.00%

16	1200.000	0.76x	4.75%
32	1250.000	0.73x	2.28%

Questions

1. *Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?*

Sixteen software threads do not create sixteen physical cores. Once all physical cores are saturated, additional threads must share execution resources. Hyper-Threading/SMT shares existing pipeline resources rather than doubling execution units. Furthermore, context switching, scheduling, and thread management add non-negligible overhead.

2. *Why was the synchronized version in Part 2 slower than running on one single core?*

Every successful hit forces all threads to contend for the same critical section to increment the counter. This causes severe lock contention and cache-invalidation traffic across CPU cores, forcing threads to spend most of their execution time stalled waiting for memory bus locks rather than executing actual computation.